Experiment 'p_4_2_c_cumulative_totals_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_c_cumulative_totals_array
Natural language statement: Write a method that takes a sequence of integers and returns cumulative totals. For example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].
Method signature: p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

Ensures

- `totals.Length == inputs.Length`
- `forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns cumulative totals. For
    ↪ example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].

The signature should be:

p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

The method should respect the following contract:

requires 0 < inputs.Length, ensures totals.Length == inputs.Length, ensures forall i :: 0
    ↪  <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i -
    ↪ 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme8v8f95SahOYIM4e8KSAmwXgf0Q
Model: gpt-5.5-2026-04-23
Created at: 1780486425
Finish reason: stop
Usage: 428 tokens in, and 660 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Final Program

```
method p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)
       requires 0 < inputs.Length
       ensures totals.Length == inputs.Length
       ensures forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs
           ↪ [0] else totals[i - 1] + inputs[i])
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 428
Output tokens: 660
Reasoning tokens: 512
Sum of 'total tokens': 1088

Experiment Timings

Overall Experiment started at 1780486425389, ended at 1780486443180, lasting 17791ms (17.79 seconds)
Iteration #1 started at 1780486425390, ended at 1780486443180, lasting 17790ms (17.79 seconds)